

Document de synthèse — Session 1 : Création de votre premier agent IA

1. Votre direction de carrière

Rôle cible recommandé : AI Product Manager / AI Product Analyst

Votre profil est déjà à 70% celui d'un PM. Vous écrivez des specs, vous gérez le lifecycle de projets, vous bridgez business et tech, vous animez des formations. Ce qui vous manquait : le langage IA et des projets concrets à montrer.

En Allemagne, ce rôle est très demandé en anglais, bien rémunéré (70-100k€+), et difficile à remplacer par l'IA car il requiert du jugement humain, de la communication et de la stratégie.

2. L'agent construit : Job Offer Analysis Agent

Ce que fait l'agent

Un agent IA qui compare des offres d'emploi avec votre profil professionnel et génère un rapport structuré.

Workflow complet

Étape 0 – Onboarding CV

L'agent lit votre profil et crée un résumé structuré

↓

Étape 1 – Recherche automatique (à construire)

L'agent scrape LinkedIn, Indeed, Xing...

↓

Étape 2 – Shortlist automatique (à construire)

Score de match avec votre profil

↓

Étape 3 – Validation humaine 🖐️ (à construire)

Vous acceptez ou rejetez chaque poste

↓

Étape 4 – Analyse approfondie ✅ (construit)

Gaps, points forts, mots-clés manquants

↓

Étape 5 – Plan d'action ✅ (construit)

CV adapté, lettre de motivation, angle LinkedIn

↓

Output – Rapport structuré par poste

Ce qui est fait vs ce qui reste

- **Fait**  : Étapes 0, 4, 5 — onboarding, analyse, rapport
 - **À faire (session 2)** : Étapes 1, 2, 3 — scraping, shortlist, validation
-

3. Concepts clés appris

Frontend vs Backend

- **Frontend** : ce que l'utilisateur voit et avec quoi il interagit (HTML, CSS, JavaScript dans le navigateur)
- **Backend** : le serveur invisible qui fait le vrai travail — appels API, base de données, logique métier
- **Pourquoi les backends existent** : le navigateur ne peut pas appeler directement des APIs externes pour des raisons de sécurité (CORS)

CORS (Cross-Origin Resource Sharing)

Règle de sécurité des navigateurs qui empêche un fichier HTML d'appeler un serveur externe non autorisé. Solution : un serveur intermédiaire (backend) qui fait le relais.

Navigateur → Serveur local Node.js → API Anthropic

Sans backend, le navigateur bloque l'appel avec "Failed to fetch".

Un serveur

Un programme qui tourne en permanence, écoute les requêtes, et y répond. En local, c'est votre `server.js`. En production, c'est la même chose mais sur des machines dans le cloud.

Tokens

Unité de texte que le LLM utilise pour lire et écrire. Ce n'est pas exactement un mot.

Texte	Tokens
"chat"	1 token
"implémentation"	3 tokens
Mot long et rare	4-6 tokens

Chaque échange avec un LLM = tokens entrants (votre message) + tokens sortants (la réponse). Les deux sont facturés séparément. Les tokens sortants coûtent plus cher car les générer demande plus de calcul.

Pour votre agent : environ 0.01-0.03\$ par analyse.

Clé API

Badge d'accès personnel à un service. Chaque appel API inclut votre clé pour identifier votre compte et facturer la consommation.

3 règles importantes :

1. **Privée** — ne jamais la partager ou la mettre sur GitHub
2. **Payante** — chaque appel coûte quelques centimes
3. **Révocable** — désactivable en un clic depuis la console

Parsing

L'action de lire et interpréter un texte pour en extraire une structure exploitable.

```
javascript

// Avant parsing – juste du texte
'{"score":71,"company":"Delivery Hero"}'

// Après parsing – un objet JavaScript utilisable
{ score: 71, company: "Delivery Hero" }
```

`JSON.parse()` fait ce travail dans votre code.

Human-in-the-loop (HITL)

Pratique professionnelle standard dans les systèmes IA : un humain valide les décisions importantes avant que l'agent continue. Dans votre agent, c'est l'étape 3 où vous acceptez ou rejetez chaque offre de la shortlist.

Pourquoi c'est important :

- L'IA ne connaît pas vos vraies priorités (réputation entreprise, ville, salaire)
- Évite les analyses approfondies sur des postes que vous auriez rejetés en 10 secondes
- Standard en entreprise pour les systèmes IA responsables

System prompt

Les instructions permanentes données à l'agent qui définissent son comportement, son rôle, et sa logique. C'est le "cerveau" de l'agent.

Context window

Limite maximale de tokens qu'un LLM peut traiter en une seule fois. Claude Sonnet : ~200 000 tokens.

4. Structure du code HTML

Votre fichier `agent-fr.html` contient 3 choses dans un seul fichier (single-file) :

HTML — le squelette

Structure de la page. Zones de texte, boutons, cartes de résultat.

```
html

<div class="card">
  <textarea id="cv-input"></textarea>
  <button onclick="submitCV()">Enregistrer</button>
</div>
```

CSS — le design

Apparence visuelle. Couleurs, tailles, espacements, arrondis. Dans la balise `<style>`.

```
css

.card {
  background: #fff;
  border-radius: 12px;
}
```

JavaScript — la logique

Ce qui fait agir la page. Appels API, validation, affichage des résultats. Dans la balise `<script>`.

Logique globale en 4 étapes :

- 1. Validation — vérifier que la zone de texte n'est pas vide
- 2. Appel API — envoyer profil + offre à Claude via `fetch()`
- 3. Parsing — interpréter la réponse JSON de Claude
- 4. Affichage — construire les cartes de résultat dynamiquement

5. Stack technique utilisée

Composant	Technologie	Rôle
Interface	HTML + CSS + JavaScript	Frontend
Serveur local	Node.js (<code>server.js</code>)	Backend / résolution CORS
IA	Claude API (<code>claude-sonnet-4-5</code>)	Analyse des offres

Composant	Technologie	Rôle
Éditeur de code	VS Code	Développement
Serveur de dev	Live Server (extension VS Code)	Aperçu en temps réel
Versioning	Git 2.50.1	Suivi des modifications

6. Outils installés

- **VS Code** — éditeur de code
- **Live Server** — extension VS Code pour aperçu en temps réel
- **Node.js v24.15.0** — environnement JavaScript backend
- **npm v11.12.1** — gestionnaire de packages Node.js
- **Git 2.50.1** — versioning et collaboration
- **Claude Code** — à installer lors de la session 2

7. Fichiers du projet

```
job-agent/  
├─ agent-fr.html      Version française de l'agent  
├─ agent-en.html      Version anglaise de l'agent  
├─ server.js          Serveur backend Node.js (résout le CORS)  
└─ README.md          Documentation bilingue du projet
```

8. Comment lancer l'agent en local

```
bash  
  
# 1. Ouvrir le Terminal et naviguer vers le dossier  
cd Desktop/job-agent  
  
# 2. Lancer le serveur backend  
node server.js  
  
# 3. Ouvrir le navigateur sur  
http://localhost:3000
```

9. Ce qu'on fera lors de la session 2

- Publier le projet sur **GitHub** pour votre portfolio
 - Installer et utiliser **Claude Code**
 - Construire les **étapes 1, 2, 3** du workflow (scraping, shortlist, validation)
 - Sécuriser le `server.js` (cacher la clé API dans un fichier `.env`)
 - Commencer le **deuxième agent** (créateur de contenu social)
-

10. Conseils pour votre portfolio

- Mentionnez le concept **Human-in-the-loop** — ça montre que vous comprenez la conception responsable des agents IA
 - Expliquez pourquoi vous avez choisi un **backend séparé** — ça montre que vous comprenez l'architecture
 - Le fait de construire cet agent **pendant votre recherche d'emploi** est en soi un argument dans votre lettre de motivation pour un poste d'AI PM
 - Préparez une démo de 2-3 minutes avec une vraie offre d'emploi
-

Document créé lors de la Session 1 — Mai 2026 Prochaine session : GitHub, Claude Code, étapes 1-2-3 du workflow